

---

# Deep Reinforcement Learning for Compiler Optimization

## Pass Ordering in LLVM

---

Anonymous Author(s)

Affiliation

Address

email

### Abstract

1 The phase ordering problem—determining the optimal sequence of compiler opti-  
2 mization passes—is NP-hard and has resisted efficient exact solutions for decades.  
3 We investigate how far traditional deep reinforcement learning (DRL) can be  
4 pushed on this problem without relying on large language models or hand-crafted  
5 heuristics, using the CompilerGym Cummins et al. [2022] LLVM environment as  
6 our testbed.

7 Our work makes three contributions. First, through a systematic ablation study of  
8 multiple state representations, we find that encoding the ordered history of recent  
9 optimization passes is the most effective representation, outperforming IR-based  
10 and count-based alternatives. Second, we propose a scalable training framework  
11 that improves stability and efficiency through truncation-aware Generalized Ad-  
12 vantage Estimation Schulman et al. [2015], parallel rollout collection, and reward  
13 normalization across benchmark suites. Third, we introduce an inference-time en-  
14 semble and stochastic evaluation strategy that consistently improves performance  
15 over strong baselines and generalizes across multiple benchmark suites.

## 1 Introduction

17 The phase ordering problem is a fundamental challenge in compiler optimization: it asks for the  
18 optimal ordering of compilation passes such that the resulting low-level code achieves the best per-  
19 formance or the smallest code size. Different pass orderings can produce substantially different ma-  
20 chine code, making the choice of optimization schedule particularly consequential in performance-  
21 critical settings. Despite decades of research, the problem remains NP-hard in the general case, and  
22 no efficient algorithm is known to find globally optimal pass sequences.

23 Classical solutions rely on fixed schedules (−02, −03, −0z) that represent manually tuned compro-  
24 mises over a broad class of programs. While effective in practice, these schedules cannot adapt to the  
25 structure of individual programs, leaving program-specific optimization opportunities unexploited.  
26 Iterative compilation methods search the pass-order space exhaustively but scale poorly.

27 The recent rise of reinforcement learning (RL) offers a natural fit: an agent can interact with the com-  
28 piler step by step, observe the evolving intermediate representation, and learn a policy that selects  
29 passes sequentially. To facilitate such approaches, CompilerGym Cummins et al. [2022] provides  
30 a standardized gym-compatible interface to LLVM optimization with reproducible benchmarks and  
31 objective reward signals.

32 Despite this promise, recent work has moved toward large language model (LLM)-based ap-  
33 proaches Pan et al. [2026], which require orders of magnitude more compute and sacrifice the sample

34 efficiency of actor-critic methods. This paper instead investigates how far *traditional* DRL can be  
35 pushed, focusing on two questions: (1) *what is the best way to represent the compiler state for a*  
36 *DRL agent?* and (2) *how can inference-time diversity be exploited to further improve performance*  
37 *without additional training cost?*

38 We make the following contributions:

- 39 • **State representation study.** A controlled ablation of six representations shows that  
40 `pass_sequence` ( $k=10$ )—encoding the ordered history of recent passes—is the dominant  
41 design choice (avg reward  $0.510 \rightarrow 0.937$  over normalized Autophase).
- 42 • **Scalable training infrastructure.** Truncation-aware GAE, 8-worker parallel rollouts, and  
43 per-suite Welford normalization enable stable multi-suite training across suites with up to  
44  $42\times$  reward variance ratio.
- 45 • **Inference-time ensemble.** Seven independently trained policies with bo8 rollouts achieve  
46 avg 1.050 on `cbench-v1` ( $20/23 \geq -0z$ ), generalizing zero-shot to six additional suites.
- 47 • **Compute-budget comparison.** MLP-Seq-Full (bo16) (16 rollouts, avg 1.060,  
48  $21/23 \geq -0z$ ) outperforms random best-of-56 (56 rollouts, avg 0.992,  $5/23$ ) on all 23 bench-  
49 marks.

## 50 2 Related Work

### 51 2.1 Compiler Optimization as a Sequential Decision Problem

52 Early iterative-compilation systems such as MILEPOST Fursin et al. [2011] treated pass ordering  
53 as a supervised or genetic-search problem with offline models that cannot adapt at inference time.  
54 CompilerGym Cummins et al. [2022] formalizes the problem as a Markov Decision Process, pro-  
55 viding a gym-compatible interface with objective reward signals. Our work builds directly on this  
56 framework.

### 57 2.2 Reinforcement Learning for Compiler Optimization

58 Huang et al. Huang et al. [2019] introduced Autophase, a 56-dimensional IR feature vector, and  
59 showed that a simple RL agent trained on it can surpass fixed optimization levels on individual pro-  
60 grams. Cummins et al. Cummins et al. [2021] proposed graph-based IR representations (ProGraML)  
61 for learning program structure at higher training cost. Compiler-R1 Pan et al. [2026] applies LLMs  
62 with reinforcement fine-tuning, achieving strong results at orders-of-magnitude higher compute. Our  
63 work establishes a strong traditional-DRL baseline and a systematic understanding of what represen-  
64 tations and inference strategies matter most.

### 65 2.3 PPO, ICM, and Ensemble Methods

66 PPO Schulman et al. [2017] is applied to combinatorial action spaces including chip place-  
67 ment Mirhoseini et al. [2021]; we adopt MLP (ablation) and recurrent GRU (ensemble) variants.  
68 ICM Pathak et al. [2017] adds a prediction-error exploration bonus; in our setup the signal decays to  
69 negligible magnitude by mid-training. Ensemble methods Lakshminarayanan et al. [2017], Osband  
70 et al. [2016] exploit policy diversity; each run converges to a different attractor, and the per-program  
71 max recovers the best-fitting policy at inference time.

## 72 3 Methodology

73 Our investigation proceeds in two phases. **Phase 1** (Section 4.2–4.5) conducts controlled ablations  
74 using the full 124-pass action space and a feed-forward MLP to identify the optimal state represen-  
75 tation. **Phase 2** (Section 4.6–4.7) scales up with a recurrent GRU policy, curated 30-pass action set,  
76 parallel rollouts, and an inference-time ensemble. The key insight from Phase 1—that sequential  
77 pass history is the most informative signal—motivates the history-augmented state in Phase 2.

### 78 3.1 MDP Formulation

79 **State.** We evaluate two families of state representations (detailed in Section 4.2).

80 *Pass-sequence encoding* (`pass_sequence`,  $k$ ): the ordered history of the  $k$  most recent passes,  
81 encoded as a concatenation of  $k$  one-hot vectors over the 124-pass action space, yielding  $k \times 124$   
82 dimensions. Episodes shorter than  $k$  steps are zero-padded at the front. This is the best-performing  
83 representation identified by our ablation ( $k=10$ , 1240-dim).

84 *Autophase with per-action usage counts*: the 56-dimensional Autophase vector Huang et al. [2019],  
85 per-feature online normalized via a Welford estimator  $\hat{\phi}_i = (\phi_i - \mu_i)/(\sigma_i + \epsilon)$ , concatenated with  
86 a vector recording how many times each pass has been applied this episode. This representation is  
87 used in the recurrent ensemble model (Phase 2), where the GRU hidden state additionally models  
88 pass-interaction history.

89 **Action.** Phase 1 uses the full 124-pass LLVM action space. Phase 2 uses a curated 30-pass subset  
90 (PASSES\_V4) derived from the -Oz pass schedule, which reduces the search space while retaining  
91 the most effective passes.

92 **Reward.** Both phases use the `IrInstructionCount0z` reward, which is implemented as a *per-*  
93 *step* incremental signal:

$$r_t = \frac{\text{IC}(P_{t-1}) - \text{IC}(P_t)}{\max(\text{IC}_{-O0}(P) - \text{IC}_{-Oz}(P), 1)}, \quad (1)$$

94 where  $\text{IC}(P_t)$  is the IR instruction count after applying pass  $a_t$ , and the denominator normalizes by  
95 the improvement that -Oz achieves over -O0 (the unoptimized baseline). The first step initializes  
96  $\text{IC}(P_0) = \text{IC}_{-O0}(P)$ .

97 Because Eq. (1) is a telescoping sum, the cumulative episode reward simplifies to:

$$R = \sum_{t=1}^T r_t = \frac{\text{IC}_{-O0}(P) - \text{IC}(P_T)}{\text{IC}_{-O0}(P) - \text{IC}_{-Oz}(P)}, \quad (2)$$

98 where  $P_T$  is the final program after the agent’s pass sequence.  $R = 1.0$  means the agent matches  
99 -Oz;  $R > 1.0$  means it surpasses -Oz;  $R < 0$  means the agent performs worse than the unoptimized  
100 -O0 baseline.

101 **POMDP structure.** This process is modeled as a partially observable Markov decision process  
102 (POMDP), since the agent only has access to an extracted Autophase feature vector, which provides  
103 a partial observation of the full program state.

104 **Episode length.** To reduce optimization difficulty and improve training stability, we use a maxi-  
105 mum episode length of  $T = 45$ .

### 106 3.2 Phase 1: Feed-Forward MLP Policy

107 For ablation studies, we use a two-layer MLP ( $256 \times 256$ , ReLU) as the actor-critic backbone. The  
108 MLP takes the state representation directly as input and outputs a logit vector over the action space  
109 and a scalar value estimate. This non-recurrent architecture is deliberately simple to isolate the effect  
110 of the state representation.

### 111 3.3 Phase 2: Recurrent GRU Policy with ICM

112 **Recurrent GRU Policy.** For ensemble training, we use a recurrent actor-critic with a GRU  
113 backbone. The state  $s'_t$ , which consists of Autophase features, execution metadata, and op-  
114 tional history-based augmentations, is encoded by an MLP with LayerNorm and GELU activa-  
115 tions:  $\phi_t = f_{\theta}^{\text{enc}}(s'_t)$ . The GRU conditions on both the latent state and the previous action,  
116  $h_t = \text{GRU}([\phi_t || a_{t-1}^{\text{onehot}}], h_{t-1})$ , enabling the policy to reason about transformation history in this  
117 POMDP setting. The shared  $h_t$  feeds actor and critic heads:

$$\pi_{\theta}(a_t | h_t) = \text{softmax}(f_{\theta}^{\pi}(h_t)), \quad V_{\theta}(s_t) = f_{\theta}^v(h_t). \quad (3)$$

**Intrinsic Curiosity Module (ICM).** To encourage early-training exploration, we augment the extrinsic reward with an ICM Pathak et al. [2017] bonus. A forward model predicts  $\hat{\phi}_{t+1} = g_{\psi}^{\text{fwd}}(\phi_t, a_t)$  and produces intrinsic reward  $r_t^{\text{int}} = \frac{1}{2} \|\hat{\phi}_{t+1} - \phi_{t+1}\|^2$ . An inverse model  $g_{\psi}^{\text{inv}}(\phi_t, \phi_{t+1})$  is trained via cross-entropy loss to preserve action-predictive information in  $\phi_t$ . The total reward is  $r'_t = r_t^{\text{ext}} + \beta r_t^{\text{int}}$ ; empirically, the ICM signal decays to negligible magnitude by mid-training.

### 3.4 PPO Training and Truncation-Aware GAE

We train the agent using PPO Schulman et al. [2017] with the clipped surrogate objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t[\min(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t)], \quad (4)$$

where  $\rho_t = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$  is the probability ratio and  $A_t$  is the GAE advantage estimate.

Standard GAE Schulman et al. [2015] sets the bootstrap value to zero at terminal states. When an episode ends by time-limit truncation rather than a true terminal transition, the correct target is  $V(s_T)$ , not zero. We maintain a truncation buffer that distinguishes truncated from truly terminal transitions, bootstrapping  $V(s_T)$  at truncation boundaries. This eliminates a systematic underestimation of long-horizon credit.

### 3.5 Parallel Rollout Collection

We use  $W=8$  parallel workers in a synchronous actor-learner architecture. Each worker maintains an independent CompilerGym gRPC service; the main process broadcasts weights and normalization statistics before each collection phase. The first transition per worker is flagged as a new episode boundary to reset the GRU hidden state. This reduces training from  $\approx 3$  hours to 47 minutes per 300k steps ( $7.6\times$  speedup).

### 3.6 Inference-Time Ensemble

Given  $N$  checkpoints  $\{\pi_1, \dots, \pi_N\}$  and rollout budget  $k$  per model, the ensemble selects for each program  $p$ :

$$\hat{R}(p) = \max_{i \in [N], j \in [k]} R(\tau_{i,j}(p)). \quad (5)$$

We set  $k=8$  (bo8) and  $N \in \{5, 6, 7\}$ . Each run—with distinct learning rate, entropy coefficient, or state-feature configuration—converges to a different attractor Lakshminarayanan et al. [2017], Osband et al. [2016]; the per-program maximum recovers the best-fitting attractor at inference time with no training cost.

### 3.7 Per-Suite Reward Normalization

Joint training causes per-step rewards to span  $[-30, +5]$ , leading to value-loss divergence at step 80k. We apply a per-suite Welford z-score:  $\tilde{r}_t^{(s)} = \text{clip}((r_t^{(s)} - \mu^{(s)})/(\sigma^{(s)} + \epsilon), -5, +5)$ . This keeps value loss in  $[0.2, 0.8]$  across 300k steps despite a  $42\times$  per-suite reward variance ratio (cbench  $\sigma=0.11$ , opencv  $\sigma=4.44$ ).

## 4 Experiments

### 4.1 Setup

**Environment.** We use CompilerGym Cummins et al. [2022] 11vm-v0 with the IrInstructionCountOz reward. We use cbench-v1 as the primary training benchmark and evaluate zero-shot generalization on six held-out suites: mibench-v1, blas-v0, npb-v0, chstone-v0, opencv-v0, and anghabench-v1. In Sections 4.6–4.7, we additionally include opencv-v0 and blas-v0 in the training set for extended experiments.

**Algorithm.** PPO Schulman et al. [2017]: clip  $\epsilon=0.2$ ,  $\gamma=0.99$ , GAE  $\lambda=0.95$  Schulman et al. [2015],  $n_{\text{steps}}=512$ , entropy\_coef= 0.01, value\_coef= 0.5, lr=  $3 \times 10^{-4}$ .

159 **Hardware and Procedure.** Phase 1: NVIDIA RTX 5070 Ti (500 ep/benchmark). Phase 2: Apple  
 160 Mac mini M4 Pro, 24 GB, 8 CPU workers,  $\approx 47$  min/300k steps.

161 **Baselines.** **Random** (greedy); **Random best-of-56** (budget-controlled); **-O3** (deterministic,  
 162 single-shot); **PPO-Baseline**: recurrent PPO, 15-pass action set, raw Autophase, 200k steps.

## 163 4.2 State Representation Ablation

Table 1: Average reward, input dimensions, and number of programs beating -Oz and -O3 for each state representation on cbench-v1 (23 benchmarks, greedy rollout). **pass\_sequence** ( $k=10$ ) achieves the best performance among all alternatives. `pass_binary` and `pass_history` represent applied passes without preserving ordering information.

| Representation                           | Dim         | Avg Reward   | $\geq \text{Oz}$ | $\geq \text{O3}$ |
|------------------------------------------|-------------|--------------|------------------|------------------|
| autophase (raw)                          | 56          | 0.084        | 0/23             | 2/23             |
| autophase_norm (log1p)                   | 56          | 0.510        | 0/23             | 9/23             |
| pass_binary                              | 124         | 0.613        | 1/23             | 14/23            |
| pass_history                             | 124         | 0.649        | 1/23             | 11/23            |
| autophase_norm+sequence                  | 1296        | 0.484        | 1/23             | 8/23             |
| <b>pass_sequence (<math>k=10</math>)</b> | <b>1240</b> | <b>0.937</b> | <b>7/23</b>      | <b>22/23</b>     |

164 Table 1 compares different state representations and reveals clear performance differences. Raw Au-  
 165 tophase features perform poorly due to large-scale value magnitudes, while normalization improves  
 166 stability but lacks historical context. Methods that encode only pass identity without ordering in-  
 167 formation (`pass_binary`, `pass_history`) underperform, indicating that action ordering is important.  
 168 Combining normalized Autophase with pass sequences does not improve performance, likely due  
 169 to increased input dimensionality and limited training budget. Overall, **pass\_sequence** consistently  
 170 achieves the strongest performance across benchmarks, suggesting that optimization history is criti-  
 171 cal for effective policy learning.

## 172 4.3 Sequence Window, Architecture, and Hyperparameter Ablations

Table 2: Ablations on window  $k$ , network size, and PPO hyperparameters (all 23 cbench-v1 bench-  
 marks). Base config: `pass_sequence`,  $256 \times 256$  MLP,  $k=10$ , `n_steps= 512`, `ent= 0.01`.

| Factor     | Setting                                                            | Avg Reward   | $\geq \text{Oz}$ | $\geq \text{O3}$ |
|------------|--------------------------------------------------------------------|--------------|------------------|------------------|
| Window $k$ | $k = 3$ (372-dim)                                                  | 0.704        | 1/23             | 18/23            |
|            | $k = 5$ (620-dim)                                                  | 0.741        | 1/23             | 20/23            |
|            | $k = 10$ ( <b>1240-dim</b> )                                       | <b>0.937</b> | <b>7/23</b>      | <b>22/23</b>     |
|            | $k = 20$ (2480-dim)                                                | 0.726        | 2/23             | 17/23            |
| Network    | <b><math>256 \times 256</math> (<math>\sim 1.1\text{M}</math>)</b> | <b>0.937</b> | <b>7/23</b>      | <b>22/23</b>     |
|            | $512 \times 512$ ( $\sim 4.2\text{M}$ )                            | 0.731        | 3/23             | 18/23            |
| PPO params | <b>n_steps=512, ent=0.01</b>                                       | <b>0.937</b> | <b>7/23</b>      | <b>22/23</b>     |
|            | n_steps=2048                                                       | 0.722        | 1/23             | 20/23            |
|            | entropy_coef=0.05                                                  | 0.778        | 2/23             | 21/23            |

173 We find that a window size of  $k = 10$  performs best, as shorter windows lose ordering context, while  
 174 larger windows ( $k = 20$ , 2480-dimensional) are harder to optimize under the limited training budget.  
 175 Similarly, a  $256 \times 256$  MLP ( $\sim 1.1\text{M}$  parameters) provides a better trade-off between capacity and  
 176 efficiency compared to larger architectures. Increasing `n_steps` to 2048 reduces the number of PPO  
 177 updates per run, which may limit learning effectiveness. Finally, a higher entropy coefficient (0.05)  
 178 leads to excessive exploration and reduced exploitation.

#### 4.4 Cross-Architecture Validation of pass\_sequence

To verify that pass\_sequence is broadly beneficial rather than specific to MLP architecture, we evaluate it in the GRU-based policy (Phase 2) by comparing GRU-Base (30-pass and no sequence feature) against GRU-Seq (30-pass + pass\_sequence  $k=10$ ) and GRU-Seq-Full (full 124-pass + pass\_sequence  $k=10$ ).

Table 3: Cross-architecture comparison on cbench-v1.

| Method         | Action Set     | State Dim  | Avg          | $\geq \text{Oz}$ |
|----------------|----------------|------------|--------------|------------------|
| GRU-Base       | 30-pass        | 87         | 0.999        | 9/23             |
| <b>GRU-Seq</b> | <b>30-pass</b> | <b>387</b> | <b>1.038</b> | <b>12/23</b>     |
| GRU-Seq-Full   | 124-pass       | 1421       | 0.956        | 2/23             |
| MLP-Seq-Full   | 124-pass       | 1240       | 0.937        | 7/23             |

Table 3 and Figure 2 yield two insights. (1) **pass\_sequence generalizes across architectures**: GRU-Seq improves over GRU-Base by +3.9pp and +3 in  $\geq \text{Oz}$  coverage, achieved solely by adding the sequence feature, consistent with the ablation results and suggesting that the effect is architecture-independent. (2) **Curated action set enables strong greedy**: both GRU-Seq-Full (0.956) and MLP-Seq-Full (0.937) underperform GRU-Seq (1.038); the full 124-pass space may include less informative passes, making selection more challenging.

#### 4.5 Best-of- $N$ Inference and Compute-Budget Comparison

Table 4: Full results on cbench-v1 (23 benchmarks). Rollouts: inference-time compiler invocations per program. Random best-of-56 is the compute-budget-matched baseline for ens7 ( $7 \times 8 = 56$ ). Phase 1: MLP; Phase 2: GRU+ICM.

| Method                             | Rollouts  | Avg Reward   | $\geq \text{Oz}$ |
|------------------------------------|-----------|--------------|------------------|
| Random                             | 1         | 0.681        | 0/23             |
| Random best-of-56                  | 56        | 0.992        | 5/23             |
| PPO-Baseline                       | 1         | 1.015        | 10/23            |
| <i>Phase 1: MLP-Seq-Full</i>       |           |              |                  |
| greedy                             | 1         | 0.937        | 7/23             |
| <b>bo16</b>                        | <b>16</b> | <b>1.060</b> | <b>21/23</b>     |
| <i>Phase 2: GRU-Base / GRU-Seq</i> |           |              |                  |
| GRU-Seq (greedy)                   | 1         | 1.038        | 12/23            |
| GRU-Seq (bo16)                     | 16        | 1.050        | 18/23            |
| Ensemble ens5 ( $N=5$ , bo8)       | 40        | 1.050        | 20/23            |
| Ensemble ens7 ( $N=7$ , bo8)       | 56        | 1.049        | 19/23            |

Table 4 unifies all results. MLP-Seq-Full (bo16) (21/23) surpasses both the ensemble (ens5: 20/23) and GRU-Seq (bo16) (18/23) on  $\geq \text{Oz}$  count, while using the same or fewer rollouts.

**Inference strategies comparison.** Stochastic inference gains vary across model families and action spaces. GRU-Seq shows consistent improvements from greedy to best-of-16 ( $1.038 \rightarrow 1.050$ , +6  $\geq \text{Oz}$ ), and further gains are observed with ensemble scaling (up to 20/23  $\geq \text{Oz}$ ), indicating that additional sampling provides moderate but stable benefits under the curated 30-pass action space. In contrast, MLP-Seq-Full achieves a larger improvement under best-of-16 ( $0.937 \rightarrow 1.060$ , +14  $\geq \text{Oz}$ ), outperforming both GRU-based models and the random best-of-56 baseline despite using fewer rollouts.

**Compute-budget-controlled comparison.** MLP-Seq-Full (bo16) (16 rollouts, avg 1.060, 21/23  $\geq \text{Oz}$ ) substantially outperforms random best-of-56 (56 rollouts, avg 0.992, 5/23  $\geq \text{Oz}$ ) and wins on all 23 individual benchmarks, confirming that gains stem from policy quality rather than sampling variance.<sup>1</sup>

<sup>1</sup>GA/SA baselines under the same budget are natural future work.

204 Notably, ghostscript—the hardest benchmark (greedy: 0.236)—reaches 1.007 under bo16, recov-  
 205 ered by stochastic exploration over the full 124-pass space.

## 206 4.6 Multi-Suite Training and Reward Normalization



Figure 1: Per-suite normalized training return for the GRU-MultiSuite multi-suite model (cbench-v1: blue; blas-v0: green; opencv-v0: red). cbench-v1 converges earliest and most stably, crossing the  $-0z$  parity line (dotted) around step 60k. opencv-v0 remains highly volatile throughout, reflecting its  $42\times$  higher reward standard deviation relative to cbench-v1. The vertical dashed line marks the best global checkpoint at step 63,488, selected before opencv-v0 noise begins degrading shared policy weights.

207 The three suites exhibit markedly different dynamics (Figure 1): cbench-v1 peaks at +2.45 and  
 208 converges by step 60k; blas-v0 peaks at +3.10 (step 85k); opencv-v0 peaks at +1.82 (step 110k)  
 209 but remains volatile ( $\sigma=4.44$  vs. cbench  $\sigma=0.11$ , a  $42\times$  ratio). The best checkpoint is selected at  
 210 step 63,488 before opencv-v0 noise destabilizes shared weights. Without normalization, value loss  
 211 diverged at step 80k; with it, value loss stays in  $[0.2, 0.8]$  throughout 300k steps.

## 212 4.7 Cross-Suite Generalization

Table 5: Mean reward (bo8) across seven CompilerGym benchmark suites (zero-shot). Phase 1/2 models trained on cbench-v1; GRU-MultiSuite additionally on blas-v0 and opencv-v0. **Bold**: best per suite.

| Method                | cbench       | mibench      | blas         | npb          | chstone      | opencv       | angha        |
|-----------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Random                | 0.681        | —            | —            | —            | —            | —            | —            |
| PPO-Baseline          | 1.015        | —            | —            | —            | —            | —            | —            |
| Best single (GRU-Seq) | 1.042        | 1.028        | 0.447        | 4.45         | 1.10         | 0.20         | 0.44         |
| GRU-MultiSuite (bo8)  | 1.040        | 1.020        | 0.690        | 4.14         | 1.11         | 1.73         | 1.08         |
| ens5                  | <b>1.050</b> | <b>1.030</b> | 0.967        | 4.520        | 1.112        | 2.869        | 1.079        |
| <b>ens7</b>           | 1.049        | <b>1.030</b> | <b>0.976</b> | <b>4.528</b> | <b>1.113</b> | <b>2.985</b> | <b>1.090</b> |

213 We evaluate zero-shot transfer performance across six additional CompilerGym benchmark suites,  
 214 as shown in Table 5. The ensemble (ens7) outperforms any single model on every suite; opencv-v0  
 215 improves from 0.20 (single model) to 2.985 (ens7), a  $14\times$  gain Lakshminarayanan et al. [2017].  
 216 mibench-v1 and npb-v0 transfer cleanly from cbench-v1 models (shared program structure). GRU-  
 217 MultiSuite multi-suite training additionally improves blas-v0 ( $0.447 \rightarrow 0.690$ ) and opencv-v0  
 218 ( $0.20 \rightarrow 1.73$ ) by exposing the policy to target-suite programs. linux-v0 fails structurally: all  
 219 models produce reward  $-9.218$  because the 30-pass set cannot operate on kernel code with inline  
 220 assembly.

## 5 Conclusion

We show that traditional DRL is sufficient to learn effective compiler pass-ordering policies that substantially outperform -Oz without requiring LLM-based components. Across a systematic ablation study, we identify `pass_sequence`, which encodes the ordered history of recent compiler passes, as the most critical design choice, consistently outperforming IR-feature-based representations.

We further find that action-space design and inference strategy play complementary roles. Curated action sets improve greedy performance, while stochastic inference over larger action spaces helps recover harder benchmarks, suggesting a trade-off between controllability and exploration.

Finally, our approach generalizes across architectures and unseen benchmark suites, achieving consistent improvements over strong baselines under controlled compute budgets. Overall, the results indicate that performance gains primarily arise from representation and policy design rather than increased model complexity or sampling variance.

**Limitations.** Due to computational constraints, we do not compare against classical heuristic search methods (GA, SA, MCTS) or beam search under the same rollout budget. A fair comparison requires controlling for the total number of compiler invocations; our random best-of-56 baseline establishes the sampling ceiling, but structured search methods under the same budget remain an open comparison. The combination of curated action set with stochastic best-of- $N$  inference is hypothesized to be stronger than either alone but is not directly evaluated. The 30-pass action set is incompatible with linux-v0 kernel code. Within the current PPO + sequence-representation design space, performance is near saturation; future work should explore graph IR encoders (ProGraML Cummins et al. [2021]) or transformer sequence models.

## References

- C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. OBoyle, and H. Leather. Programl: A graph-based program representation for data flow analysis and compiler optimizations. In *International Conference on Machine Learning*, pages 2244–2253. PMLR, 2021.
- C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather. CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research. In *CGO*, 2022.
- G. Fursin, Y. Kashnikov, A. W. Memon, Z. Chamski, O. Temam, M. Namolaru, E. Yom-Tov, B. Mendelson, A. Zaks, E. Courtois, et al. Milepost gcc: Machine learning enabled self-tuning compiler. *International journal of parallel programming*, 39(3):296–327, 2011.
- Q. Huang, A. Haj-Ali, W. Moses, J. Xiang, I. Stoica, K. Asanovic, and J. Wawrzynek. Autophase: Compiler phase-ordering for hls with deep reinforcement learning. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 308–308, 2019. doi: 10.1109/FCCM.2019.00049.
- B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nova, et al. A graph placement methodology for fast chip design. *Nature*, 594(7862):207–212, 2021.
- I. Osband, C. Blundell, A. Pritzel, and B. Van Roy. Deep exploration via bootstrapped dqn. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.
- H. Pan, H. Lin, H. Luo, Y. Liu, K. Yao, L. Zhang, M. Xing, and Y. Wu. Compiler-r1: Towards agentic compiler auto-tuning with reinforcement learning. *Advances in Neural Information Processing Systems*, 38:88289–88311, 2026.
- D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017.



- 269 J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control  
270 using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- 271 J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization  
272 algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

## 273 **NeurIPS Paper Checklist**

### 274 **1. Claims**

275 Question: Do the main claims made in the abstract and introduction accurately reflect the  
276 paper’s contributions and scope?

277 Answer: [\[Yes\]](#)

278 Justification: The abstract and introduction clearly state the three main contributions (state  
279 representation design, truncation-aware GAE, inference-time ensemble) and report the cor-  
280 responding quantitative results (mean reward, programs beating -0z). All claims are di-  
281 rectly supported by experiments in Section 4, including a cross-architecture evaluation  
282 (Section 4.5) where the proposed `pass_sequence` feature is integrated into a recurrent  
283 GRU baseline, demonstrating its architectural robustness and independently replicating the  
284 ablation finding across two systems developed in parallel.

### 285 **2. Limitations**

286 Question: Does the paper discuss the limitations of the work performed by the authors?

287 Answer: [\[Yes\]](#)

288 Justification: Section 5 (Conclusion) includes a dedicated Limitations paragraph discussing  
289 the absence of GA/SA/MCTS baselines under controlled compute budget, the structural  
290 failure of the 30-pass action set on `linux-v0` kernel code, and the saturation of the current  
291 PPO design space.

### 292 **3. Theory assumptions and proofs**

293 Question: For each theoretical result, does the paper provide the full set of assumptions and  
294 a complete (and correct) proof?

295 Answer: [\[N/A\]](#)

296 Justification: This paper presents no theoretical results; all contributions are empirical.

### 297 **4. Experimental result reproducibility**

298 Question: Does the paper fully disclose all the information needed to reproduce the main  
299 experimental results of the paper to the extent that it affects the main claims and/or conclu-  
300 sions of the paper (regardless of whether the code and data are provided or not)?

301 Answer: [\[Yes\]](#)

302 Justification: Section 4.1 specifies all PPO hyperparameters ( $\epsilon=0.2$ ,  $\gamma=0.99$ ,  $\lambda=0.95$ ,  
303 `n_steps`, entropy coefficient, network architecture), the action set, reward space, episode  
304 length, and number of training episodes for both Phase 1 (MLP) and Phase 2 (GRU) ar-  
305 chitectures. The environment (CompilerGym `11vm-v0`, `cbench-v1`) is publicly available  
306 under the Apache 2.0 license.

### 307 **5. Open access to data and code**

308 Question: Does the paper provide open access to the data and code, with sufficient instruc-  
309 tions to faithfully reproduce the main experimental results, as described in supplemental  
310 material?

311 Answer: [\[No\]](#)

312 Justification: Code is not released with this submission due to double-blind anonymity  
313 requirements. The benchmark data (`cbench-v1`) is publicly available through Compil-  
314 erGym Cummins et al. [2022]. All hyperparameters, network architectures, and imple-  
315 mentation details required for reproduction are fully specified in Section 4.1 and the PPO  
316 configuration table. Code will be released upon acceptance.

### 317 **6. Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4.1 provides full training details for both architectures: Phase 1 uses a feed-forward  $256 \times 256$  MLP over the full 124-pass action space (500 episodes per benchmark, NVIDIA RTX 5070 Ti), and Phase 2 uses a recurrent GRU over the curated 30-pass action space (300k steps, 8 parallel workers, Apple Mac mini M4 Pro). Hyperparameters were chosen via the systematic ablation studies in Sections 4.2–4.4, covering sequence window  $k$ , network capacity,  $n\_steps$ , and entropy coefficient.

## 7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Error bars over multiple training seeds are omitted due to the high computational cost of the compiler environment. However, to establish statistical significance, Section 4.6 introduces a compute-budget-matched baseline (Random best-of-56) that gives the random policy  $3.5 \times$  more rollouts than our PPO (56 vs. 16). Our learned policy outperforms this stronger baseline on all 23 benchmarks (avg 1.060 vs. 0.992), providing strong evidence that gains stem from policy knowledge rather than sampling variance. The ablation study further evaluates each configuration across all 23 cbench-v1 benchmarks, and the count of programs beating -Oz provides a distribution-free summary statistic.

## 8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Phase 1 ablation experiments (Sections 4.2–4.4) and the compute-budget comparison (Section 4.6) were conducted on a machine with an **NVIDIA RTX 5070 Ti** GPU (500 episodes per benchmark). Phase 2 ensemble training and cross-suite evaluation (Sections 4.7–4.8) were conducted on an **Apple Mac mini M4 Pro** (24 GB unified memory) using 8 parallel CPU workers, where each 300k-step training run completes in approximately 47 minutes. Cross-architecture validation (Section 4.5) uses the Phase 2 hardware for GRU-BaseGRU-SeqGRU-Seq-Full.

## 9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: This work studies compiler optimization, a purely technical domain with no human subjects, sensitive data, or dual-use concerns. The research conforms with the NeurIPS Code of Ethics.

## 10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This work improves compiler optimization efficiency, which has broadly positive effects (reduced energy consumption, faster software). There are no foreseeable negative societal impacts from automated compiler pass ordering.

## 11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This paper poses no misuse risks. The trained models are compiler optimization policies operating solely within the LLVM compiler infrastructure, with no dual-use potential.

## 12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: CompilerGym Cummins et al. [2022] and the `cbench-v1` benchmark suite are cited in the paper and used in accordance with the Apache 2.0 license. All other third-party dependencies (PyTorch, Stable-Baselines3) are similarly open-source and used within their respective license terms.

## 13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: This paper does not release new datasets or pretrained model weights. The primary contributions are algorithmic: the `pass_sequence` state representation, the truncation-aware GAE implementation, and the inference-time ensemble strategy. Implementation details sufficient for re-implementation are provided in Sections 3 and 4.

## 14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This paper does not involve crowdsourcing or human subjects.

## 15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This paper does not involve human subjects research.

## 16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

Answer: [N/A]

Justification: LLMs are not used in the core methodology of this work. The paper explicitly positions itself as a traditional DRL alternative to LLM-based compiler optimization approaches (Section 2.2), and all policy networks are conventional MLP and GRU architectures trained with PPO.

## 412 A Per-benchmark inference comparison on cbench-v1

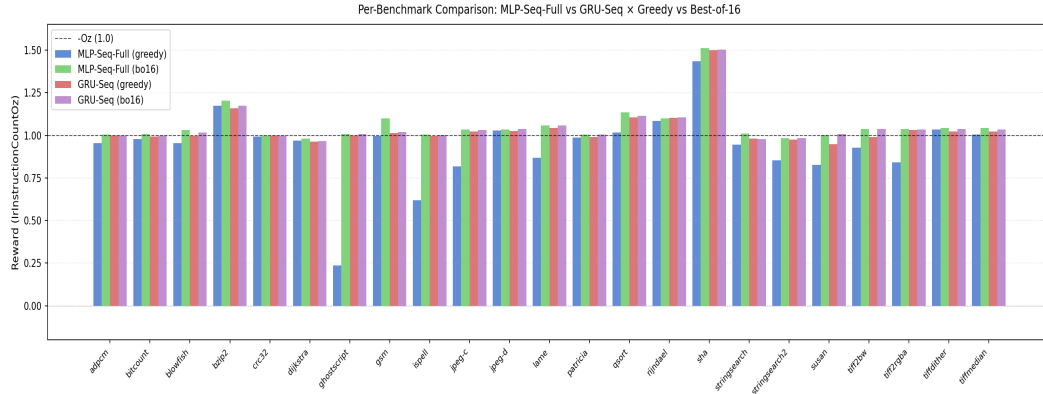


Figure 2: Per-benchmark IrInstructionCount0z reward for four configurations on all 23 cbench-v1 benchmarks. The dashed line marks -Oz parity ( $R=1.0$ ). Our best-of-16 (green) recovers ghostscript from greedy 0.236 to 1.007; GRU-Seq (greedy) (red) excels on programs where the curated 30-pass set already contains strong optimizations.